

Problema de las 8 Reinas: Análisis Comparativo de Algoritmos CSP

Pola Escobar, José Salazar

Septiembre 22, 2025

1. Introducción

El problema de las 8 reinas es un clásico problema de satisfacción de restricciones (CSP) que consiste en colocar 8 reinas en un tablero de ajedrez de 8×8 de modo que ninguna de ellas se ataque entre sí. Este problema ha sido ampliamente estudiado en el campo de la inteligencia artificial como un ejemplo paradigmático para evaluar diferentes técnicas de resolución de CSPs.

2. Formulación como CSP

2.1. Definición del Problema

El problema de las N reinas se puede formular como un CSP de la siguiente manera:

2.1.1. Variables

Cada columna del tablero representa una variable:

$$X = \{X_1, X_2, \dots, X_N\}$$

donde X_i indica la fila en la que se coloca la reina de la columna i .

2.1.2. Dominios

Cada reina puede colocarse en cualquier fila:

$$D_i = \{1, 2, \dots, N\} \quad \forall i \in \{1, 2, \dots, N\}$$

2.1.3. Restricciones

El problema tiene dos tipos de restricciones:

1. No puede haber dos reinas en la misma fila:

$$X_i \neq X_j \quad \forall i \neq j$$

2. No puede haber dos reinas en la misma diagonal:

$$|X_i - X_j| \neq |i - j| \quad \forall i \neq j$$

3. Implementaciones

Se implementaron tres algoritmos diferentes para resolver el problema:

3.1. Backtracking Básico

El algoritmo de backtracking básico explora el espacio de soluciones de manera sistemática, asignando valores a las variables una por una y retrocediendo cuando se detecta una violación de restricciones.

```
1 def es_valido(tablero, fila, col):
2     n = len(tablero)
3     for i in range(fila):
4         for j in range(n):
5             if tablero[i][j] == 1:
6                 if j == col or abs(j - col) == abs(i - fila):
7                     return False
8     return True
9
10 def backtracking(tablero, fila, recursive_call, backtrack):
11     if fila == len(tablero):
12         return tablero, recursive_call, backtrack
13
14     for i in range(len(tablero)):
15         if es_valido(tablero, fila, i):
16             tablero[fila][i] = 1
17             resultado, recursive_call, backtrack = backtracking(
18                 tablero, fila + 1, recursive_call + 1, backtrack)
19             if resultado:
20                 return resultado, recursive_call, backtrack
21             tablero[fila][i] = 0
22             backtrack += 1
23     return None, recursive_call, backtrack
```

Listing 1: Implementación de Backtracking Básico

3.2. Forward Checking

El forward checking mejora el backtracking básico propagando las restricciones hacia adelante, eliminando valores del dominio de variables no asignadas que son inconsistentes con la asignación actual.

```
1 def forward_checking(tablero, fila, dominios, recursive_call,
2   backtrack):
3     n = len(tablero)
4     if fila == n:
5         return tablero, recursive_call, backtrack
6
7     for col in dominios[fila][:]:
8         tablero[fila][col] = 1
9         nuevos = {f: list(dominios[f]) for f in dominios}
10
11        # Propagacion de restricciones
12        for f in range(fila + 1, n):
13            if col in nuevos[f]:
14                nuevos[f].remove(col)
15            diag1 = col + (f - fila)
16            diag2 = col - (f - fila)
17            if diag1 in nuevos[f]:
18                nuevos[f].remove(diag1)
19            if diag2 in nuevos[f]:
20                nuevos[f].remove(diag2)
21
22        if any(len(nuevos[f]) == 0 for f in range(fila + 1, n)):
23            tablero[fila][col] = 0
24            backtrack += 1
25            continue
26
27        resultado, recursive_call, backtrack = forward_checking(
28            tablero, fila+1, nuevos, recursive_call + 1,
29            backtrack)
30
31        if resultado:
32            return resultado, recursive_call, backtrack
33
34        tablero[fila][col] = 0
35        backtrack += 1
36
37    return None, recursive_call, backtrack
```

Listing 2: Implementación de Forward Checking

3.3. Forward Checking con MRV

La heurística MRV (Minimum Remaining Values) selecciona la variable con el menor número de valores válidos restantes en su dominio, lo que puede reducir significativamente el espacio de búsqueda.

```
1 def forward_checking_MRV(tablero, dominios, restantes,
2   recursive_call, backtrack):
3     if not restantes:
4         return [row[:] for row in tablero], recursive_call,
5             backtrack
6
7     # Seleccion MRV: variable con menor dominio
8     fila = min(restantes, key=lambda x: len(dominios[x]))
9
10    for col in dominios[fila][:]:
11        tablero[fila][col] = 1
12        nuevos = {f: list(dominios[f]) for f in dominios}
13
14        # Propagacion de restricciones
15        for f in restantes - {fila}:
16            if col in nuevos[f]:
17                nuevos[f].remove(col)
18                diag1 = col + (f - fila)
19                diag2 = col - (f - fila)
20                if diag1 in nuevos[f]:
21                    nuevos[f].remove(diag1)
22                if diag2 in nuevos[f]:
23                    nuevos[f].remove(diag2)
24
25            if any(len(nuevos[f]) == 0 for f in restantes - {fila}):
26                tablero[fila][col] = 0
27                backtrack += 1
28                continue
29
30        resultado, recursive_call, backtrack =
31            forward_checking_MRV(
32                tablero, nuevos, restantes - {fila}, recursive_call +
33                    1, backtrack)
34        if resultado:
35            return resultado, recursive_call, backtrack
36
37        tablero[fila][col] = 0
38        backtrack += 1
39
40    return None, recursive_call, backtrack
```

Listing 3: Implementación de Forward Checking con MRV

4. Experimentos y Resultados

4.1. Configuración Experimental

Se realizaron experimentos con las siguientes configuraciones:

- Tamaños de tablero: $N = 8, 10, 12, 14, 16, 18, 20$
- Repeticiones por algoritmo: 4
- Métricas evaluadas: tiempo de ejecución, llamadas recursivas, número de backtracks

4.2. Resultados Experimentales

Los experimentos se ejecutaron de forma paralela para optimizar el tiempo de cómputo. A continuación se presentan los resultados obtenidos:

4.2.1. Tiempo de Ejecución

La Figura 1 muestra la evolución del tiempo de ejecución para los tres algoritmos según el tamaño del tablero.

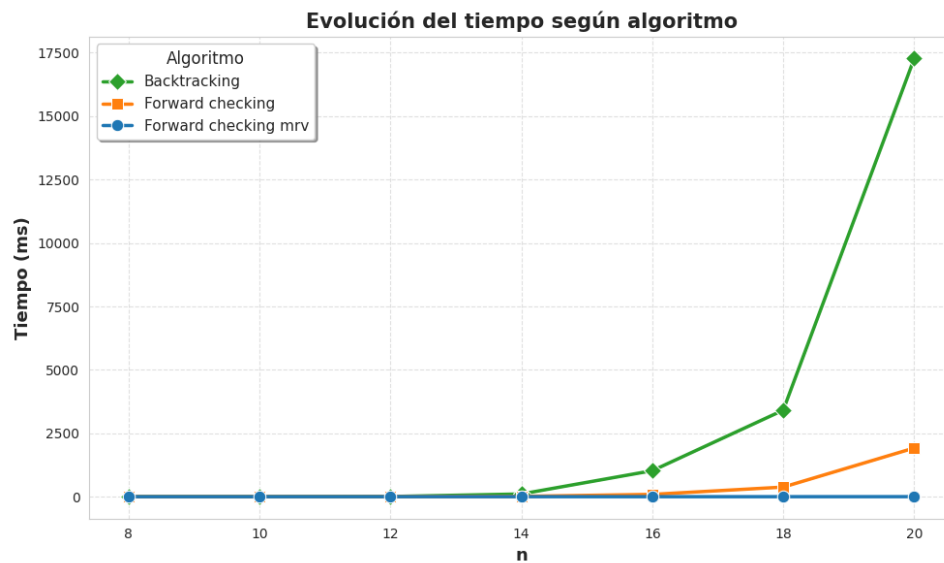


Figura 1: Evolución del tiempo de ejecución según algoritmo

4.2.2. Llamadas Recursivas

La Figura 2 presenta el número de llamadas recursivas realizadas por cada algoritmo.

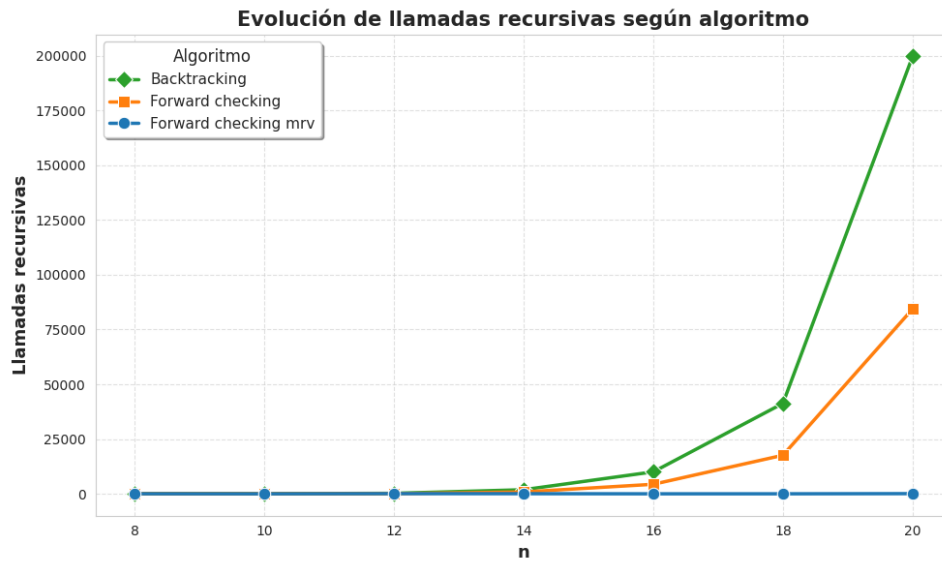


Figura 2: Evolución de llamadas recursivas según algoritmo

4.2.3. Número de Backtracks

La Figura 3 muestra el número de retrocesos realizados por cada algoritmo.

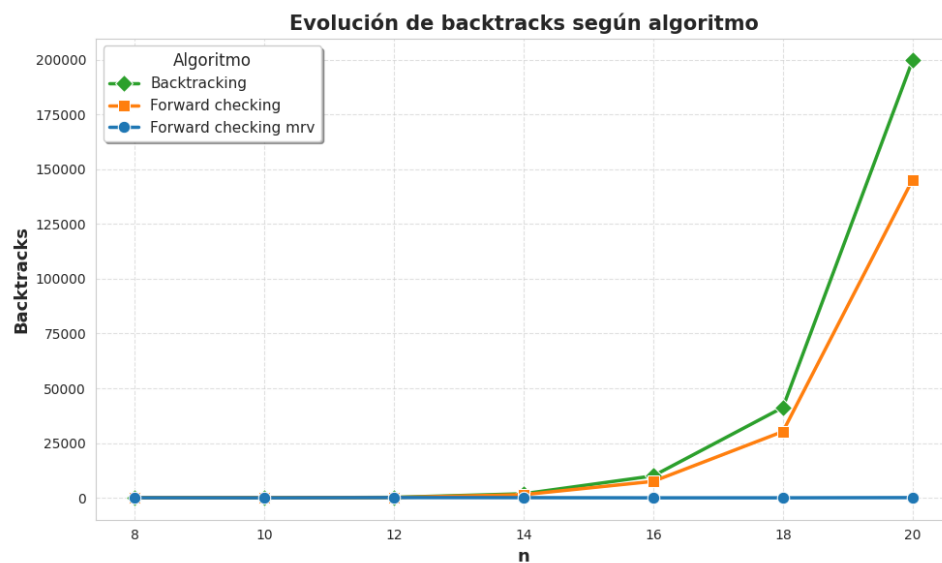


Figura 3: Evolución de backtracks según algoritmo

4.3. Análisis Comparativo

Basándose en los resultados experimentales, se pueden hacer las siguientes observaciones:

1. **Forward Checking con MRV** es el algoritmo más eficiente en términos de tiempo de ejecución y número de llamadas recursivas.
2. **Forward Checking** sin MRV muestra mejoras significativas sobre el backtracking básico, especialmente para tableros más grandes.

3. **Backtracking básico** es el menos eficiente, requiriendo más tiempo y más exploración del espacio de soluciones.
4. La diferencia de rendimiento se acentúa conforme aumenta el tamaño del tablero, demostrando la escalabilidad de las técnicas más avanzadas.

5. Análisis de Escalabilidad

Para evaluar la escalabilidad de las técnicas implementadas, se probaron tableros de diferentes tamaños ($N = 8, 10, 12, 14, 16, 18, 20$). Los resultados muestran que:

- El crecimiento del tiempo de ejecución es exponencial para todos los algoritmos, como se esperaba para un problema NP-completo.
- Forward Checking con MRV mantiene la mejor relación tiempo/tamaño del tablero.
- La diferencia de rendimiento entre algoritmos se amplifica con el aumento del tamaño del tablero.

6. Conclusiones

6.1. Reflexión sobre Eficiencia

Los resultados experimentales confirman que **Forward Checking con MRV** es la técnica más eficiente para resolver el problema de las N reinas. Esta superioridad se debe a:

1. **Propagación temprana de restricciones:** El forward checking elimina valores inconsistentes antes de explorar ramas del árbol de búsqueda.
2. **Selección inteligente de variables:** La heurística MRV reduce el factor de ramificación del árbol de búsqueda.
3. **Detección temprana de fallos:** La combinación de ambas técnicas permite detectar inconsistencias más rápidamente.

6.2. Lecciones Aprendidas

- La propagación de restricciones es crucial para mejorar la eficiencia de los algoritmos CSP.
- Las heurísticas de selección de variables pueden tener un impacto significativo en el rendimiento.
- La combinación de múltiples técnicas de optimización puede producir mejoras sinérgicas.
- La medición empírica es esencial para validar las mejoras teóricas.

7. Código Fuente

El código fuente completo del proyecto está disponible en el archivo `ochoreinas.py`, que incluye:

- Implementación de los tres algoritmos
- Sistema de logging para monitoreo
- Ejecución paralela de experimentos
- Generación automática de gráficos
- Exportación de resultados a CSV

8. Referencias

1. Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson.
2. Dechter, R. (2003). *Constraint Processing*. Morgan Kaufmann.
3. Mackworth, A. K. (1977). Consistency in networks of relations. *Artificial Intelligence*, 8(1), 99-118.